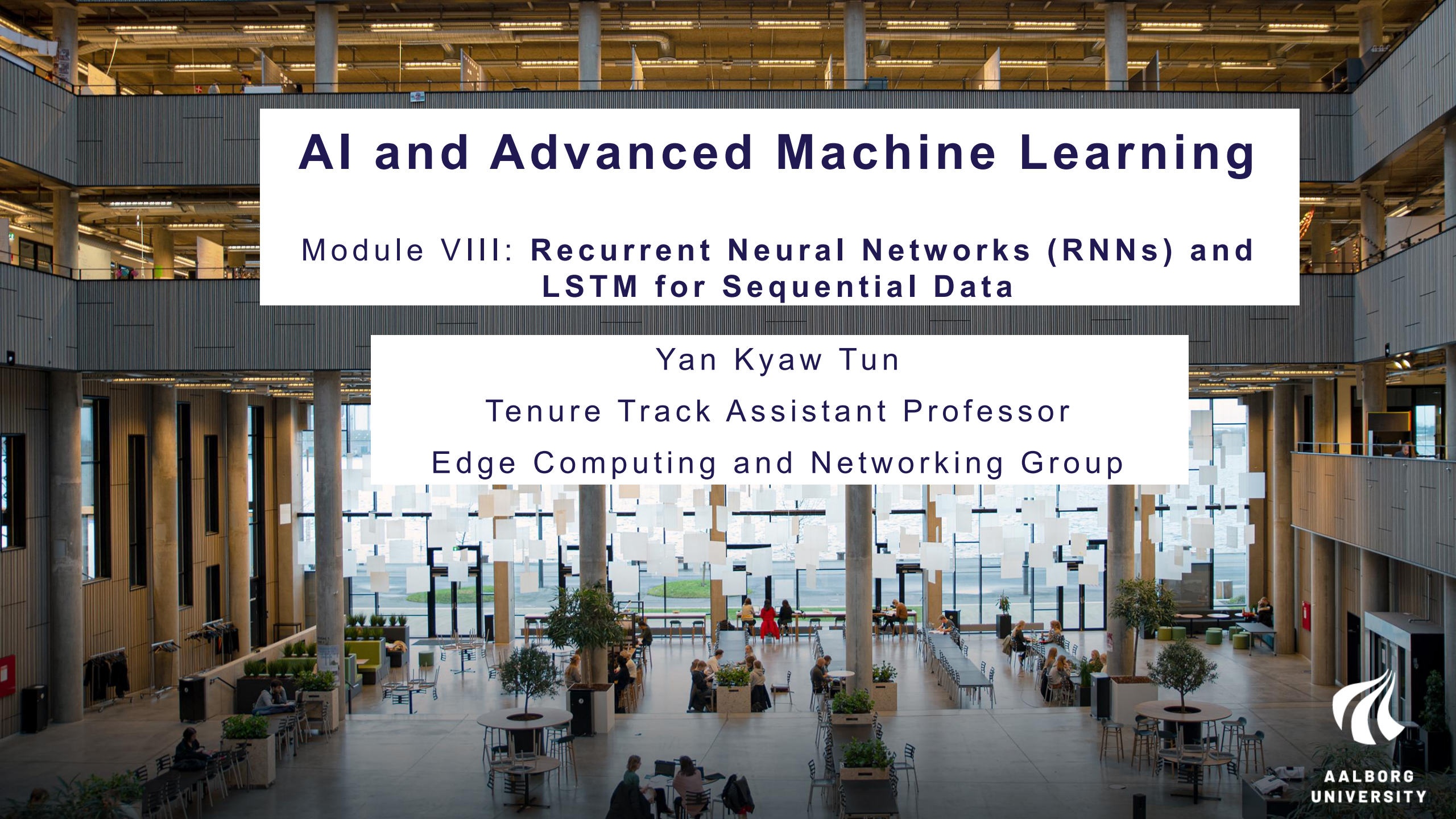


AI and Advanced Machine Learning

Module VIII: **Recurrent Neural Networks (RNNs) and LSTM for Sequential Data**

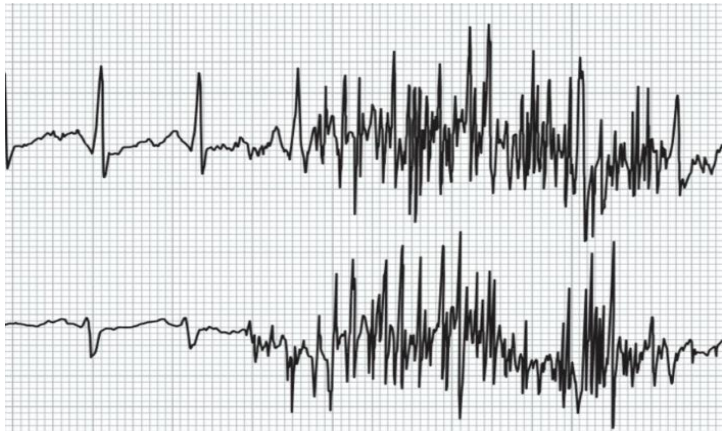
Yan Kyaw Tun

Tenure Track Assistant Professor

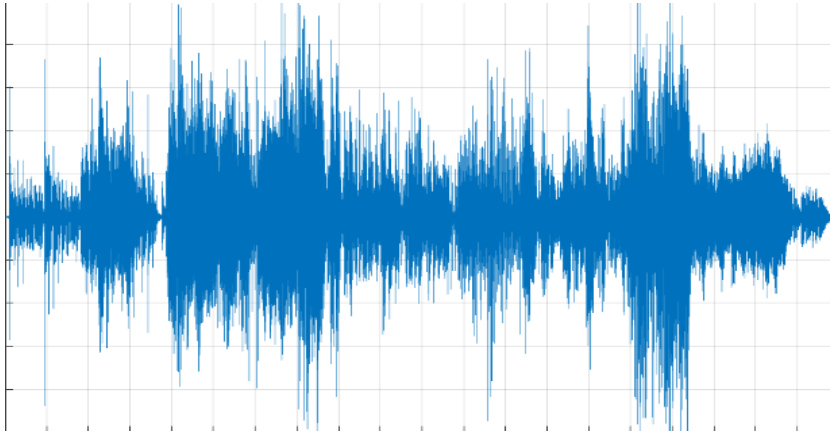
Edge Computing and Networking Group

AALBORG UNIVERSITY

Sequential Data



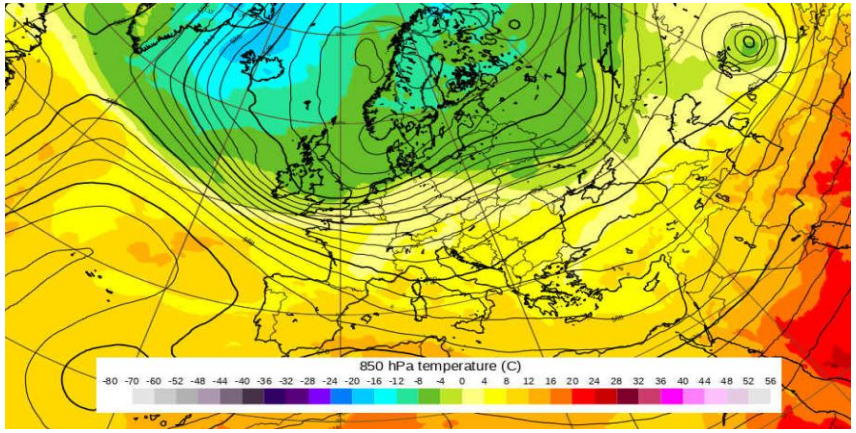
ECG



Audio

DNA Sequencing

Financial Market

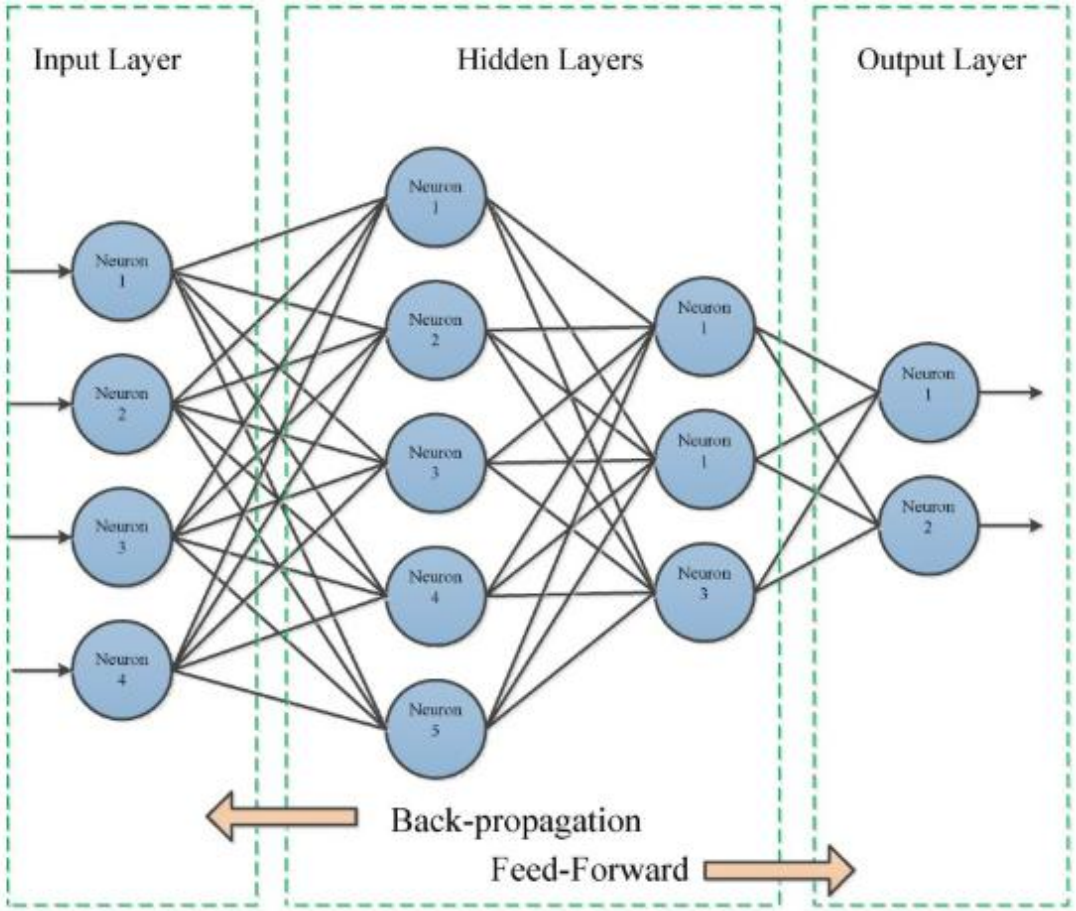


Weather

Video

Revisit Feed-Forward Neural Networks

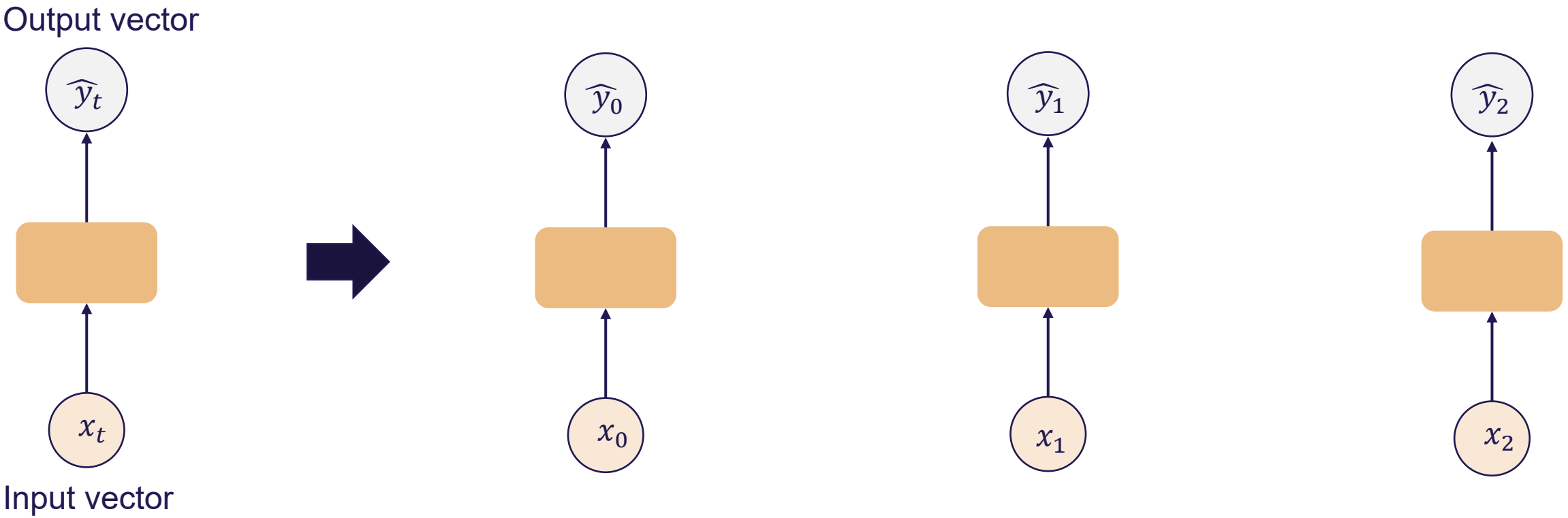
- Cannot handle sequential data
- Considers only the current input
- Cannot memorize previous inputs
- Loss the information of neighborhood
- Do not have any loops or circles



Design Requirements for Sequential Data

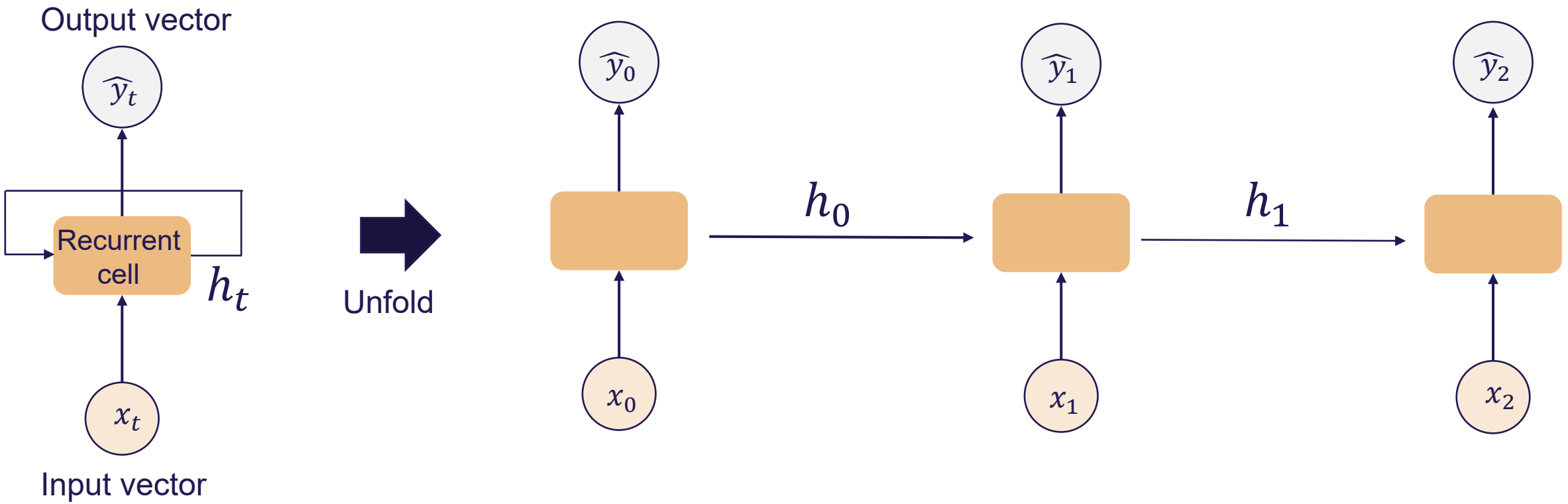
- Handle variable length sequences
- Track long-term dependencies
- Maintain information about order
- Share parameters across the sequence

Handling Individual Time Steps



$$\widehat{y}_t = f(x_t)$$

Neurons with Recurrence

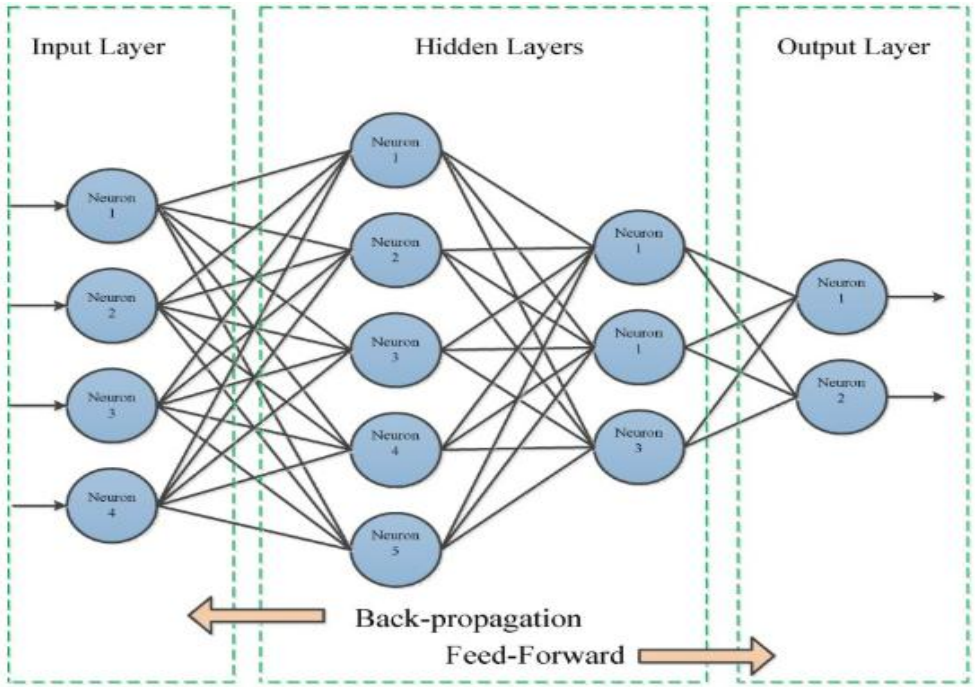


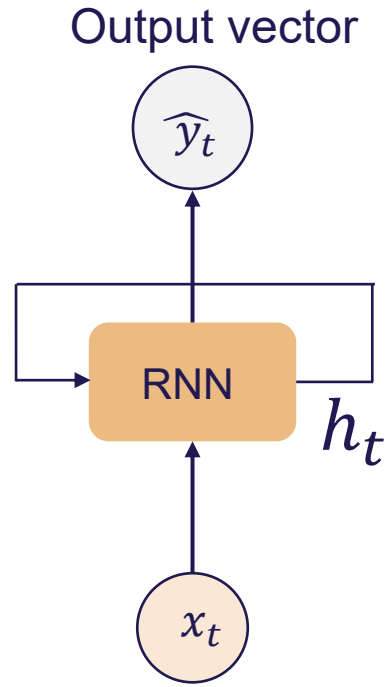
$$\hat{y}_t = f(x_t, h_{t-1})$$

output input Past memory

Recurrent Neural Networks (RNNs)

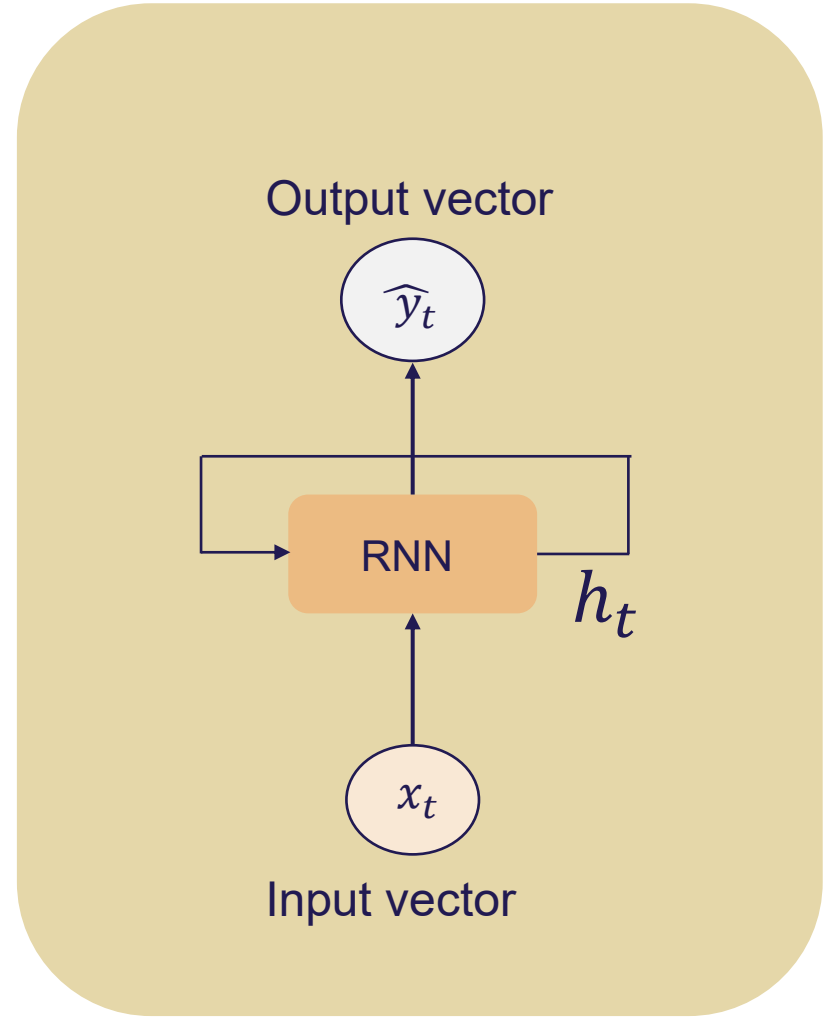
- An artificial neural network to work for time series data or sequential data
- Use a hidden layer to remember specific information about a sequence
- Have a memory that stores all information
- Formed from feed-forward networks



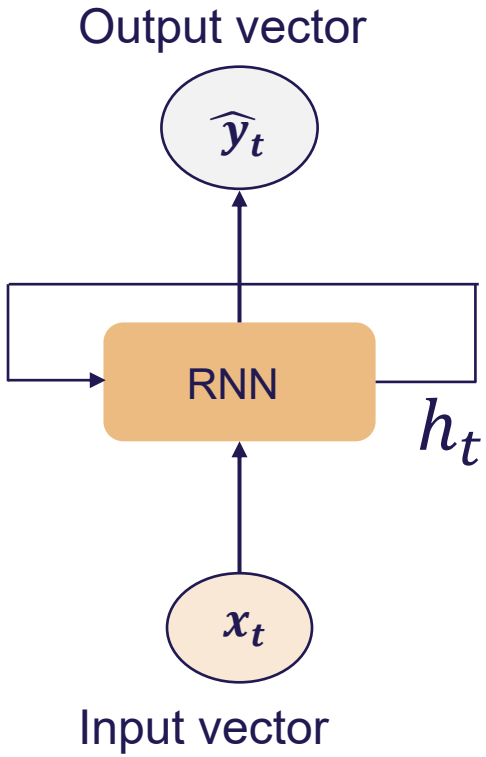


Recurrent Neural Networks (RNNs)

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



Recurrent Neural Networks (RNNs)



Output Vector

$$\hat{y}_t = \boldsymbol{W}_{\boldsymbol{hy}}^{\boldsymbol{T}} h_t$$

Update Hidden State

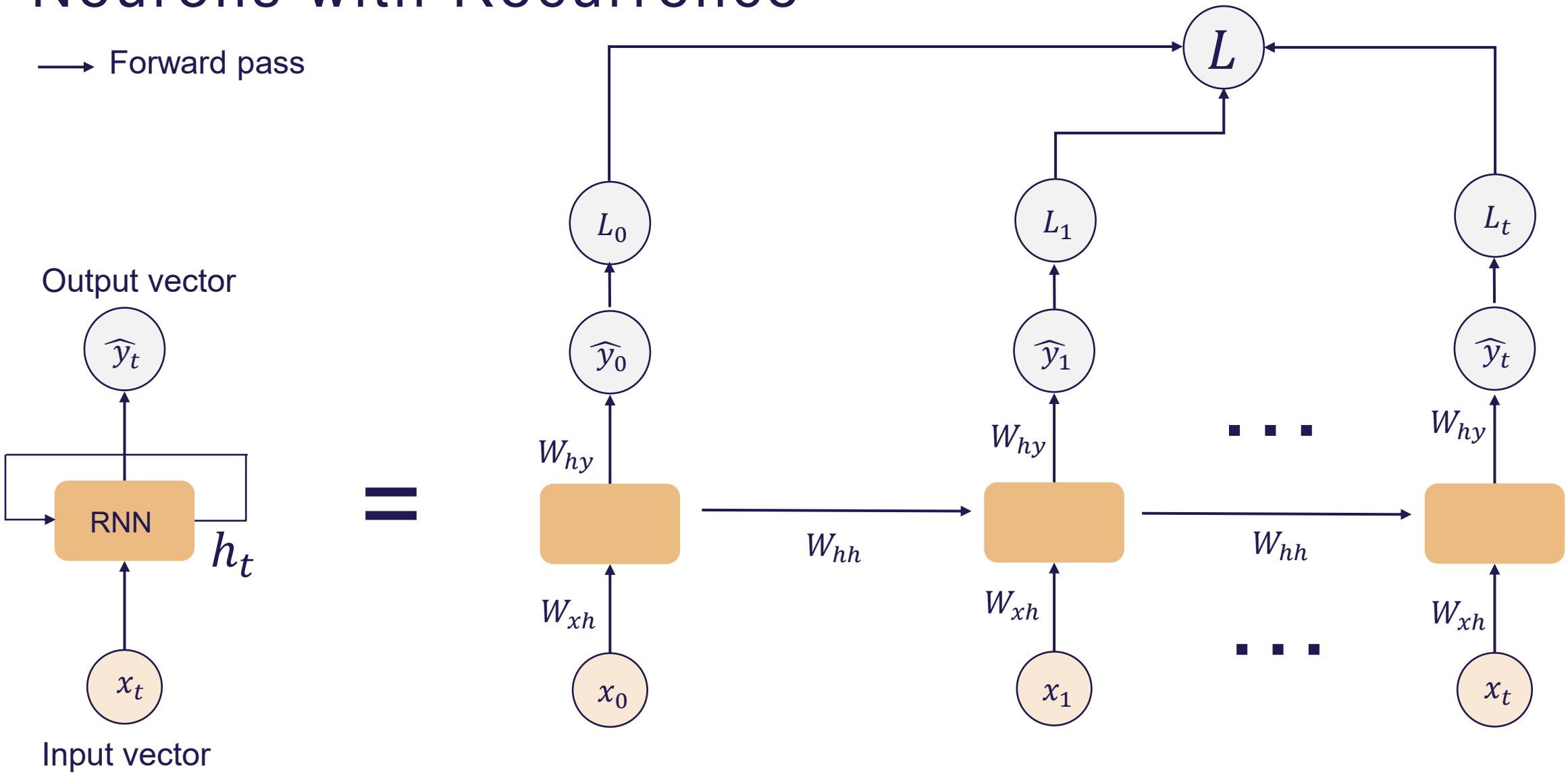
$$h_t = tanh(\boldsymbol{W}_{\boldsymbol{hh}}^{\boldsymbol{T}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}}^{\boldsymbol{T}} x_t)$$

Input vector

$$x_t$$

Neurons with Recurrence

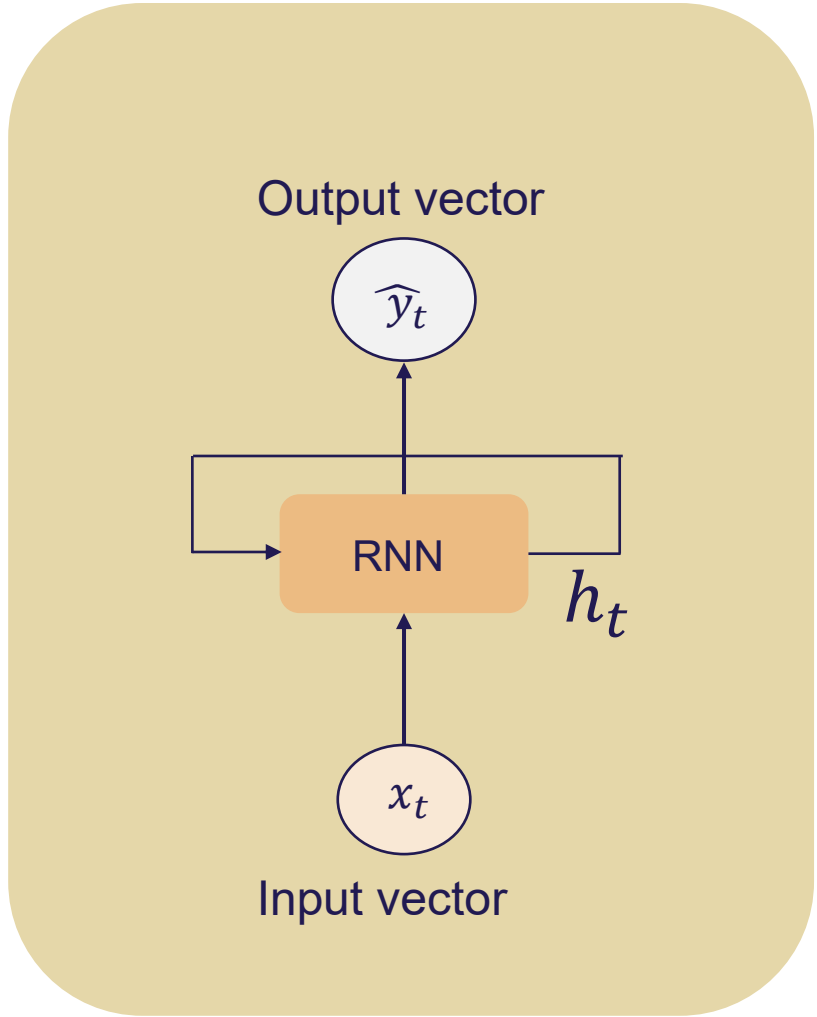
Forward pass



- We use the same weight matrices at every time step

RNN from Scratch in TensorFlow

```python
class MyRNNCell(tf.keras.layers.Layer):
  def __init__(self, rnn_units, input_dim, output_dim):
    super(MyRNNCell, self).__init__()

    # Initialize weight matrices
    self.W_xh = self.add_weight([rnn_units, input_dim])
    self.W_hh = self.add_weight([rnn_units, rnn_units])
    self.W_hy = self.add_weight([output_dim, rnn_units])

    # Initialize hidden state to zeros
    self.h = tf.zeros([rnn_units, 1])

  def call(self, x):
    # Update the hidden state
    self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )

    # Compute the output
    output = self.W_hy * self.h

    # Return the current output and hidden state
    return output, self.h
```



Simple Example

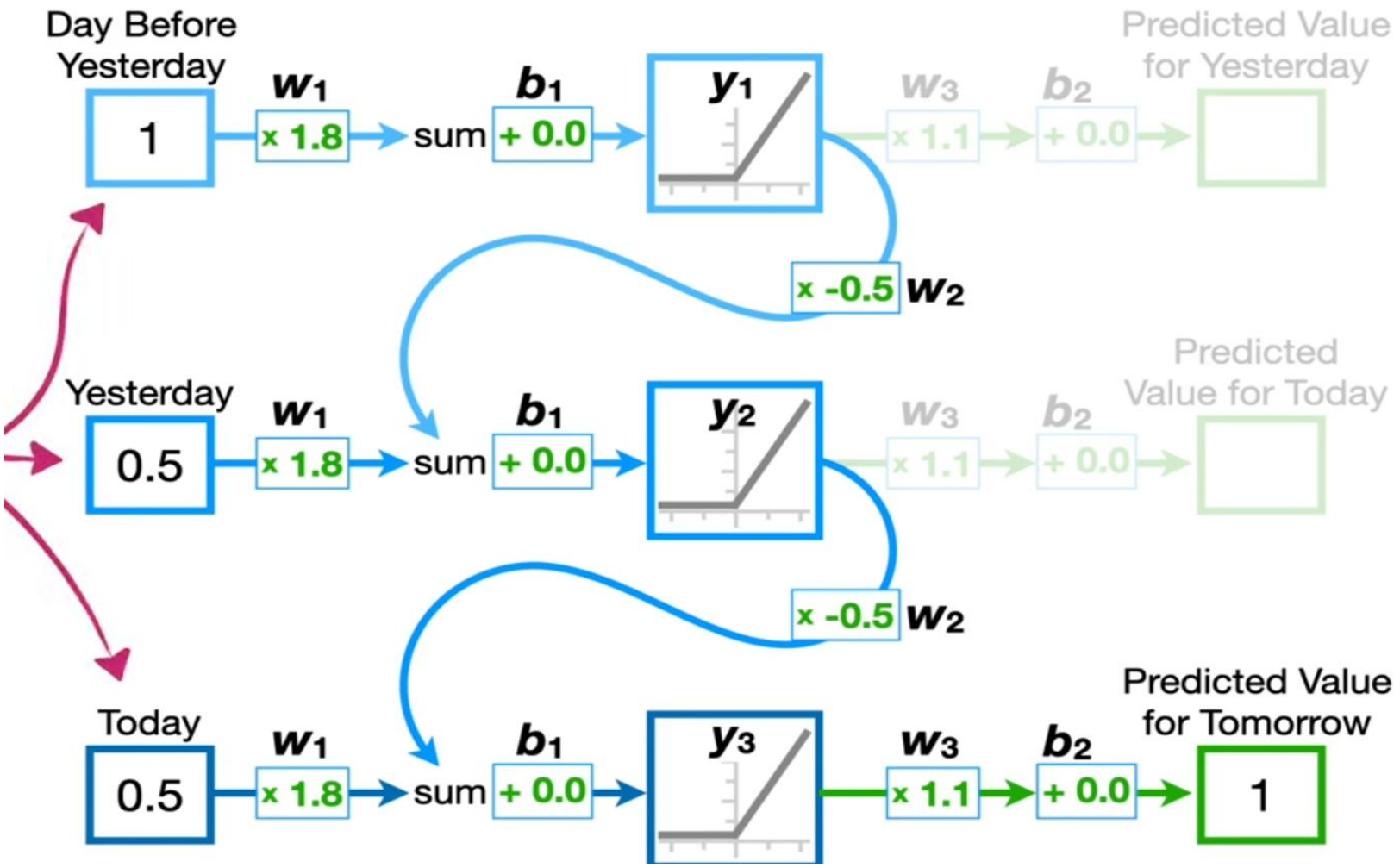


Image source: https://www.youtube.com/watch?v=AsNTP8Kwu80

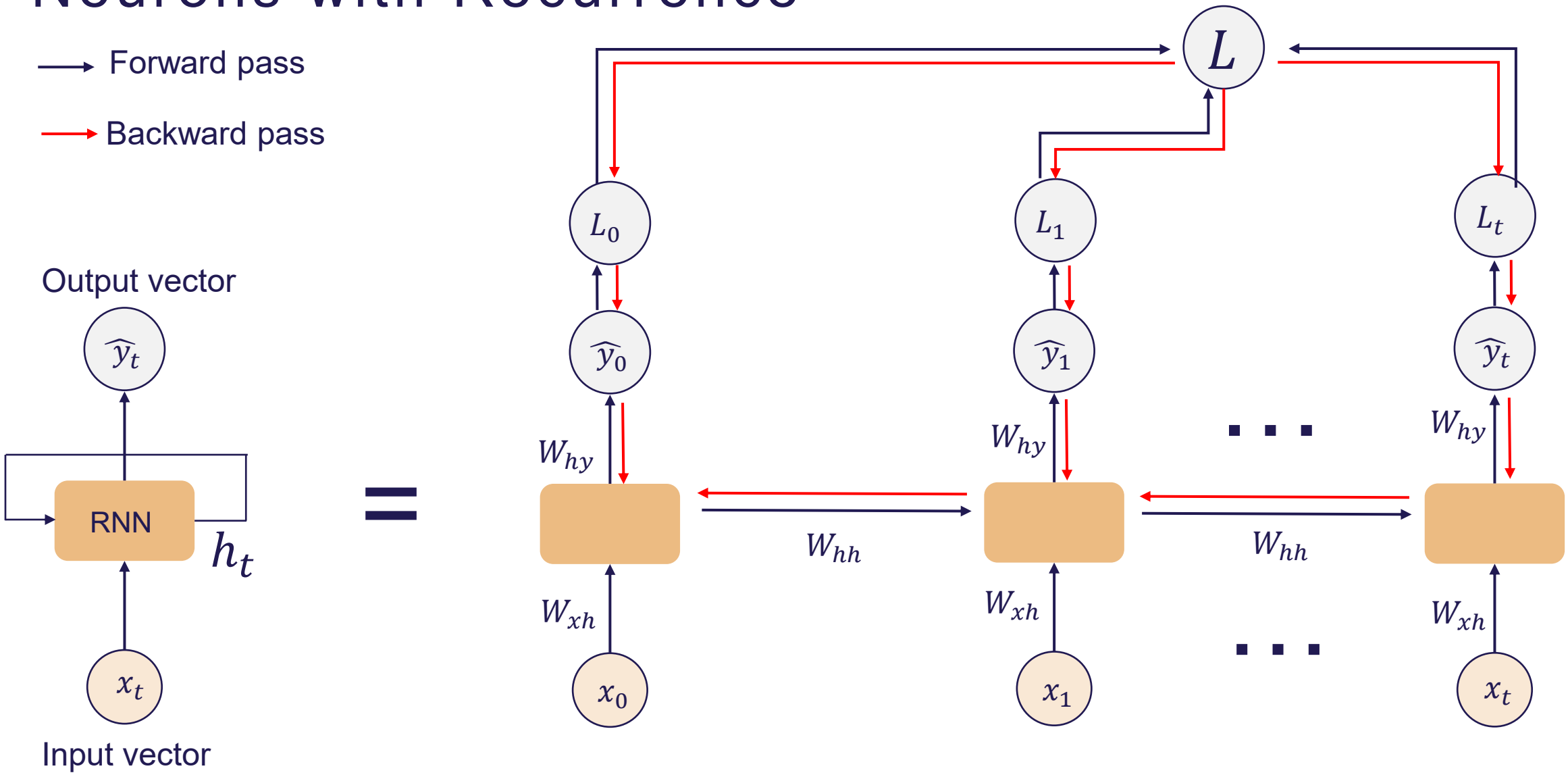
Steps for Training Recurrent Neural Network (RNN)

- The initial input is fed into the RNN using shared weights and an activation function
- The current hidden state is computed using the **current input** and the **previous hidden state (memory)**
- The hidden state at time step “t” becomes the previous state for time step “t+1”
- This process is repeated for all time steps in the sequence
- The final output is computed from the hidden state(s), depending on the task (many-to-one, many-to-many, etc.)
- An error (loss) is calculated by comparing the predicted output with the actual target output
- If there is an error, **backpropagation through time (BPTT)** is used to update the weights across all time steps

Neurons with Recurrence

Forward pass

Backward pass

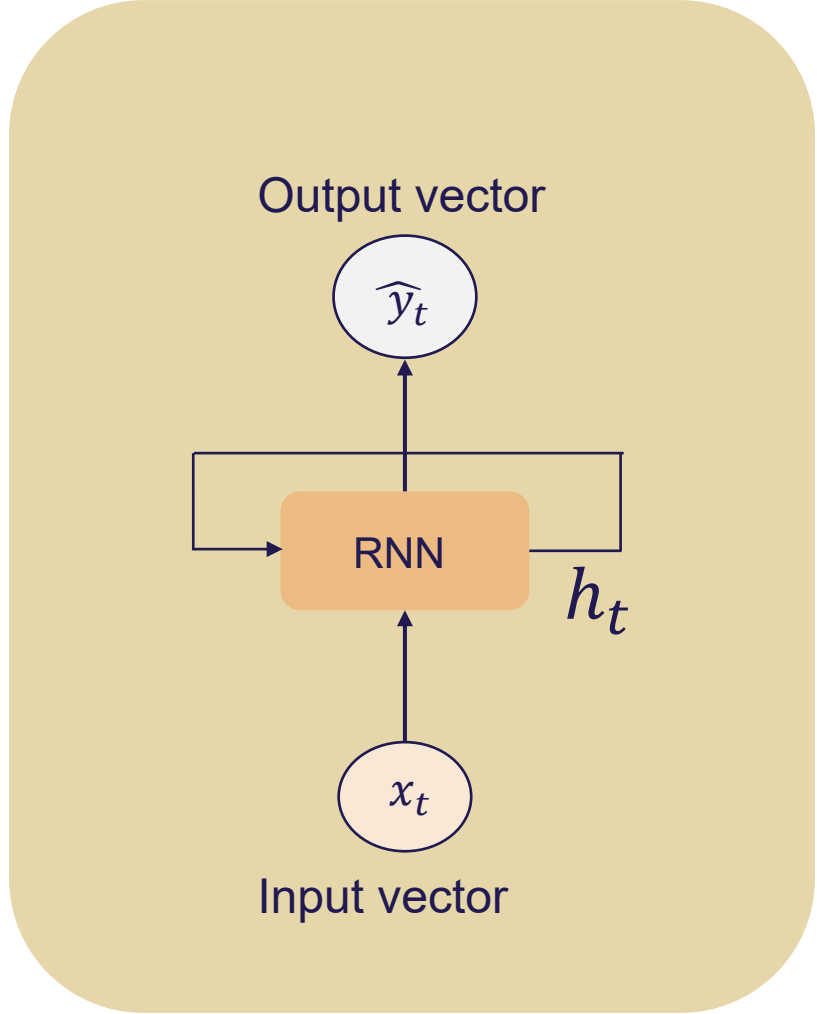


- We use the same weight matrices at every time step

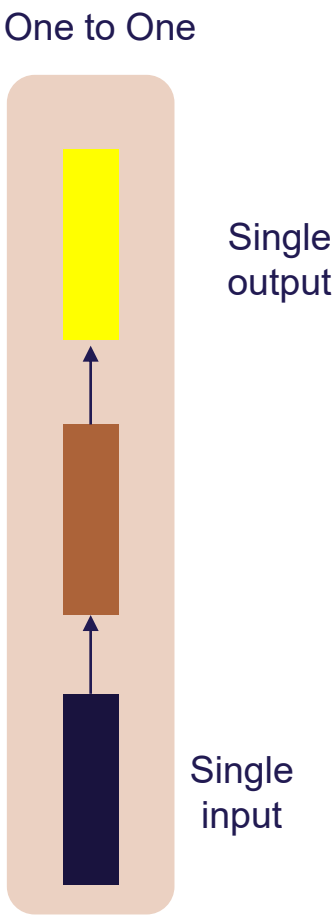
RNN Implementation: TensorFlow & PyTorch

```
from tf.keras.layers import SimpleRNN
model = SimpleRNN(rnn_units)
```

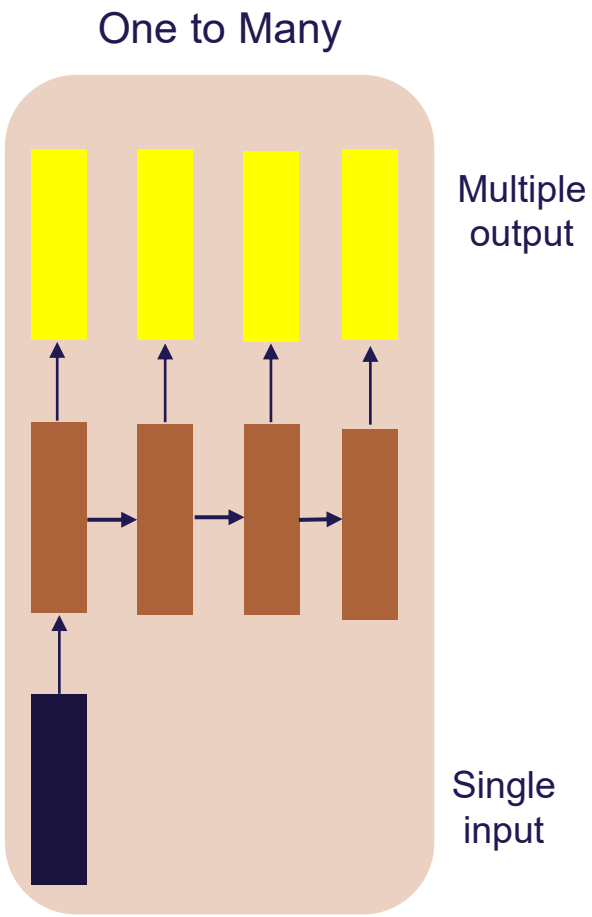
```
from torch.nn import RNN
model = RNN(input_size, rnn_units)
```



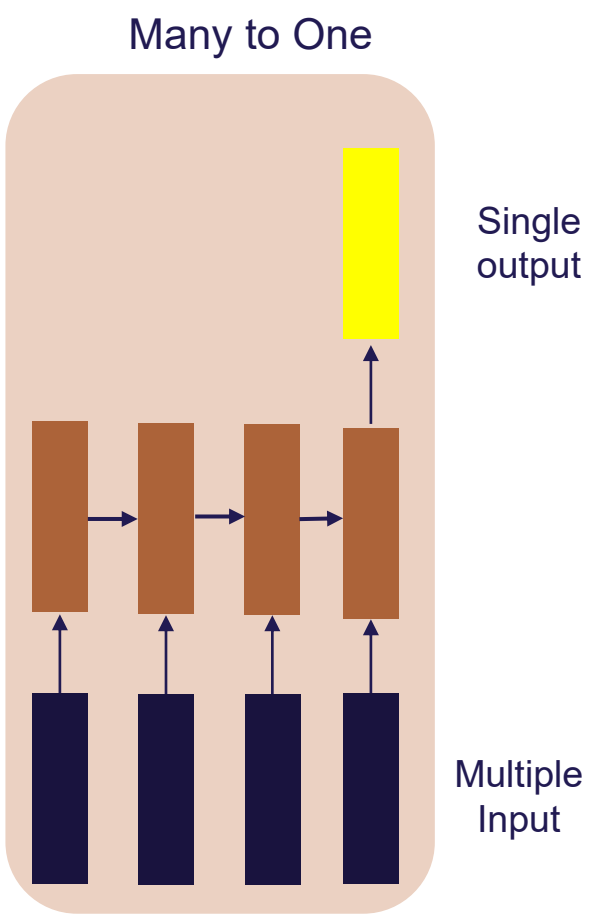
Types of Recurrent Neural Networks



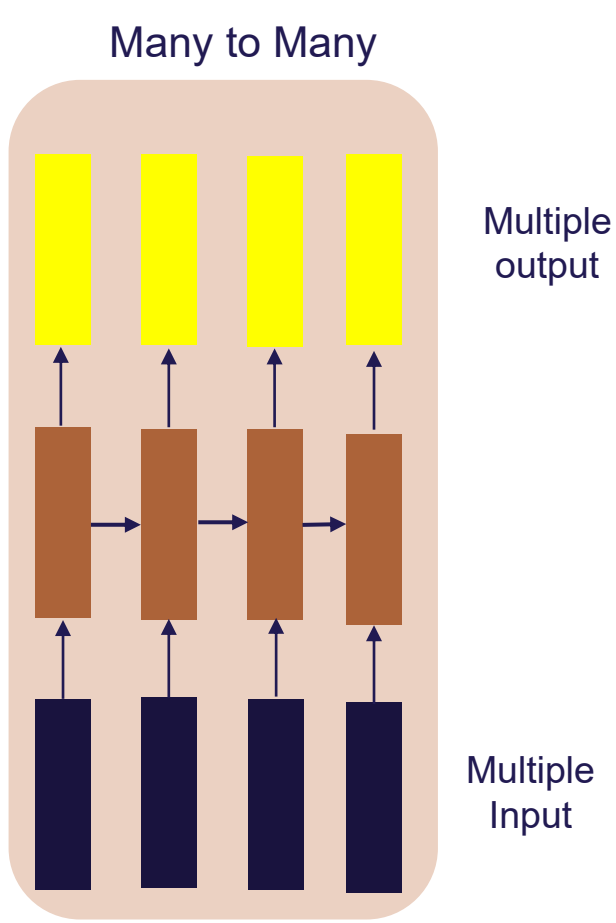
Classification, Regression



Generating **Image Caption**



Sentiment Analysis



Machine Translation

Example: Predict the Next Word Using RNN

- I love pizza
- Vocabulary: ["I", "love", "pizza"]
- One-hot encoding: I → [1, 0, 0], love → [0, 1, 0], pizza → [0, 0, 1]
- Can’t capture the relationship between word: love · like = 0, love · hate = 0
- If vocabulary = 50000 words, then vector size = 50000. Spare Vector -> waste of memory
- Word Embeddings: love → [0.21, -0.5, 0.88, ...], pizza → [0.19, -0.45, 0.90, ...] (usual size: 100/200/300)

Example: Music Generation

- Input: sheet music
- Output: Next character in sheet music
- Example: Franz Schubert – Symphony No. 8 (Unfinished) finalized by artificial intelligence

  https://www.youtube.com/watch?v=_6OUGRsslJY

Advantages and Disadvantages

Advantages:

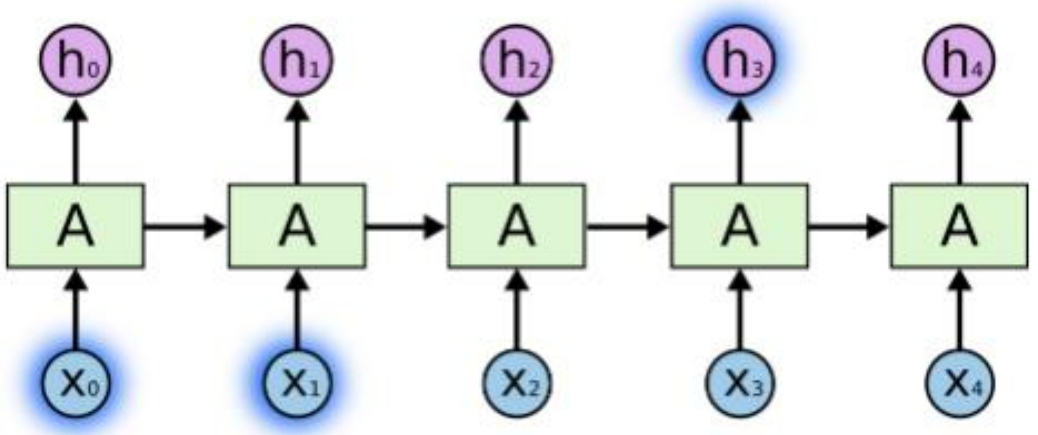
- Can process input sequences of variable length
- Capture temporal dependencies in sequential data (useful for time-series and language tasks)
- The number of model parameters does not increase with input length (fixed model size)
- Share the same parameters across all time steps, improving efficiency and generalization

Disadvantages:

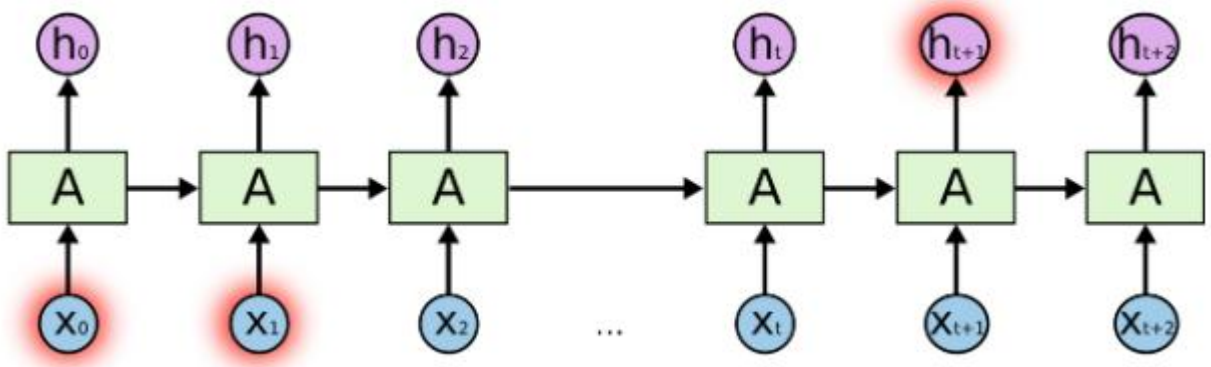
- **Difficult to learn long-term dependencies**: RNNs struggle to remember information from far back in the sequence due to vanishing gradient
- **No parallel computation over time steps**: Since each step depends on the previous one, RNNs must process sequences sequentially, making training slow
- **Slow training on long sequences**: The step-by-step nature increases computation time and limits scalability
- **Limited memory capability in practice** : Although designed to retain past information, basic RNNs often forget earlier inputs in long sequences
- **Training instability**: Gradients can become very large (explode), making optimization difficult

Long-Term Dependencies Problem in RNN

- She poured herself a cup of hot coffee/tea



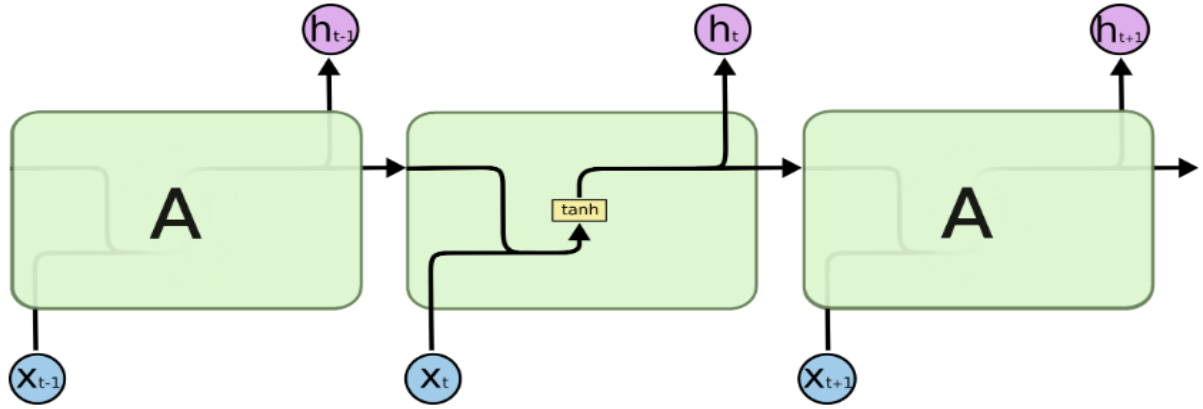
- She grew up in Italy, moved to Canada years later, studied engineering at university, and now she speaks fluent Italian



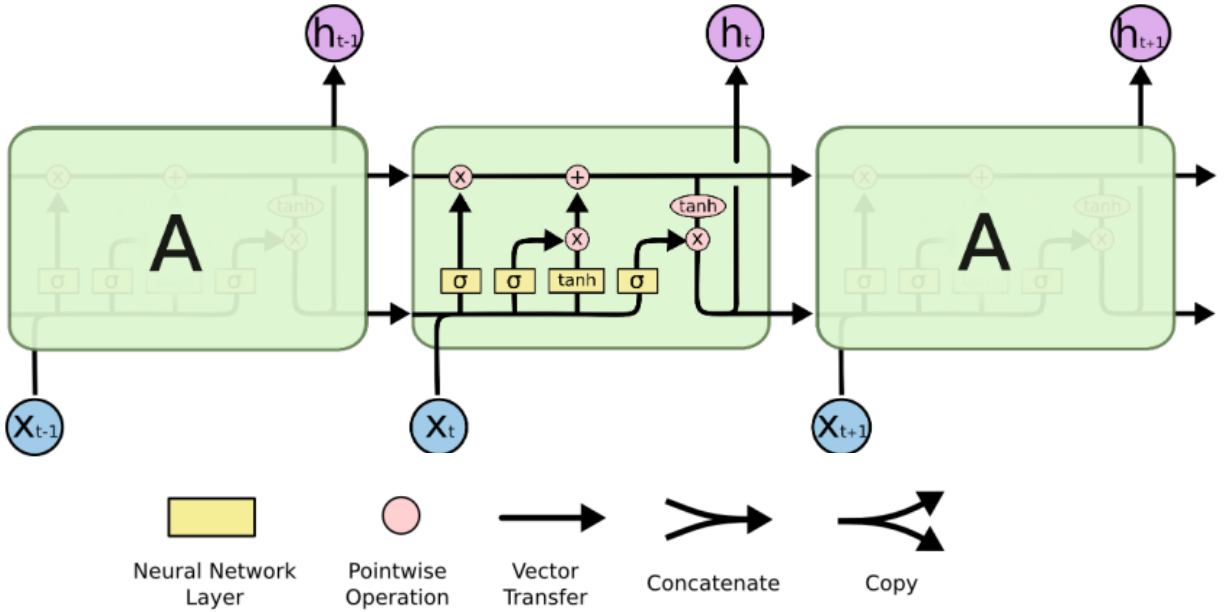
Long Short Term Memory (LSTM)

- A Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to learn long-term dependencies, and it is widely used in areas like natural language processing, speech recognition, and time series forecasting (***Proposed in 1997 by Hochreiter and Schmidhuber***)

- LSTMs are specifically built to address the long-term dependency problem. In fact, retaining information over long time spans is essentially their intended function, rather than something they find difficult to learn



Recurrent Neural Networks (RNNs)

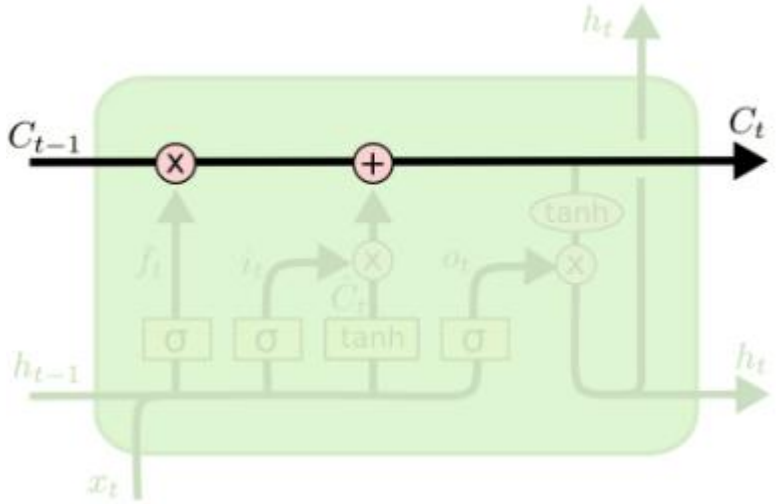


Long Short Term Memory (LSTM)

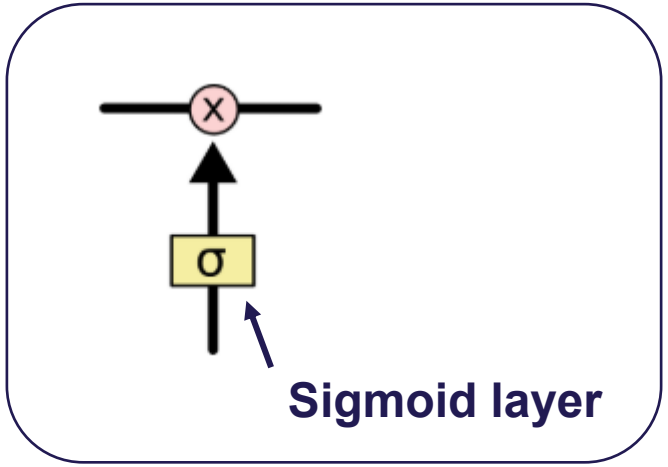
Image source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Long Short Term Memory (LSTM)

- The core idea behind LSTM is the **cell state** (**a conveyor belt**)



- The LSTM does have the ability **to remove or add information to the cell state**

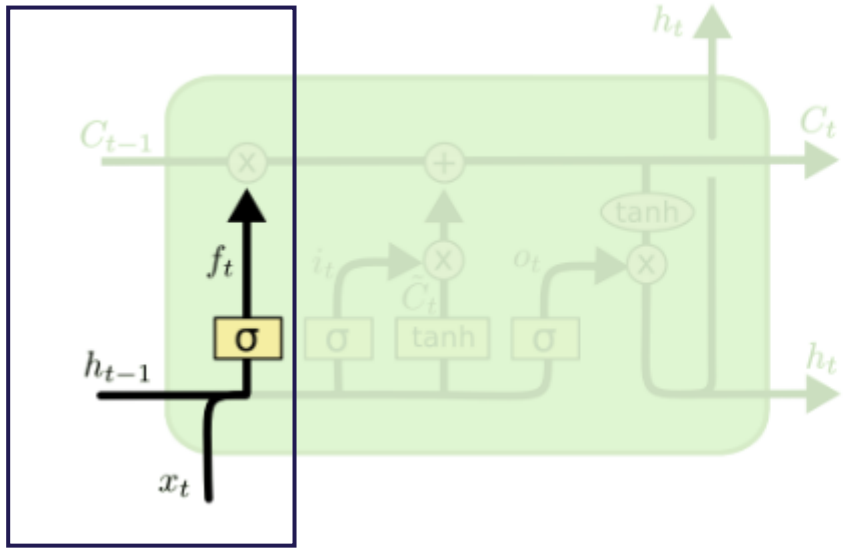


Forget Gate

Long Short Term Memory (LSTM)

- The first step in our LSTM is to decide what information we're going to throw away from the cell state

Forget Gate



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

- "0" represents "completely get rid of this"
- "1" represents "completely keep this"

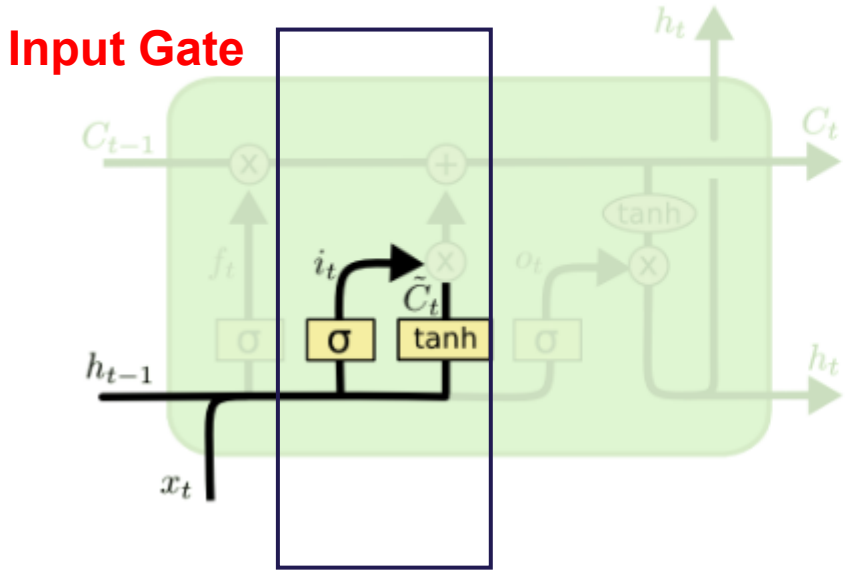
Example in language model

John went to the store. **Mary** picked up her phone."

First subject: **John**
New subject: **Mary**

Long Short Term Memory (LSTM)

- Choose what new information to add to memory



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

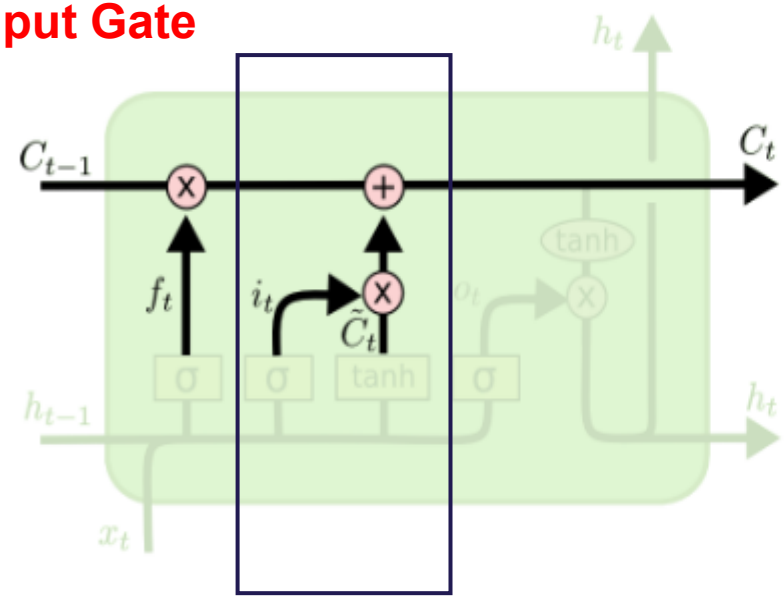
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

- i_t: How much of the new information should be written into memory
- $\widetilde{C_t}$: new candidate information (what the model might want to store now)

Long Short Term Memory (LSTM)

- Update the cell state

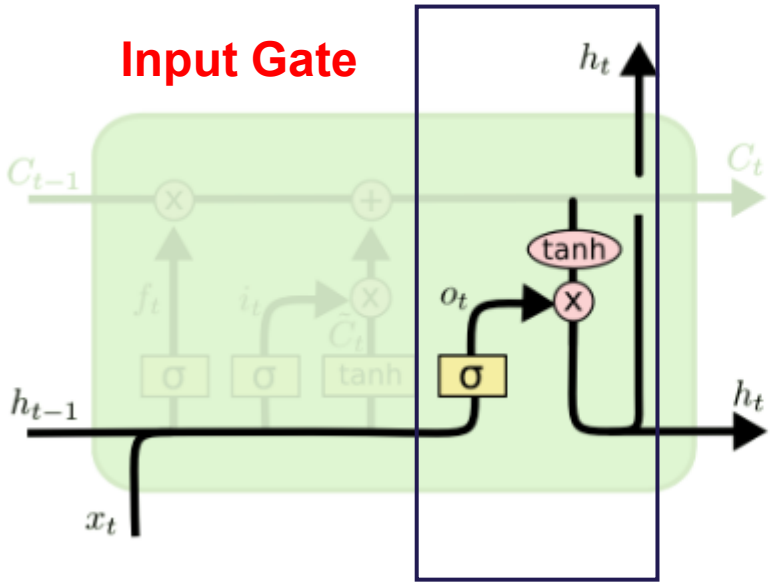
Input Gate



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Long Short Term Memory (LSTM)

- Decide what is the output



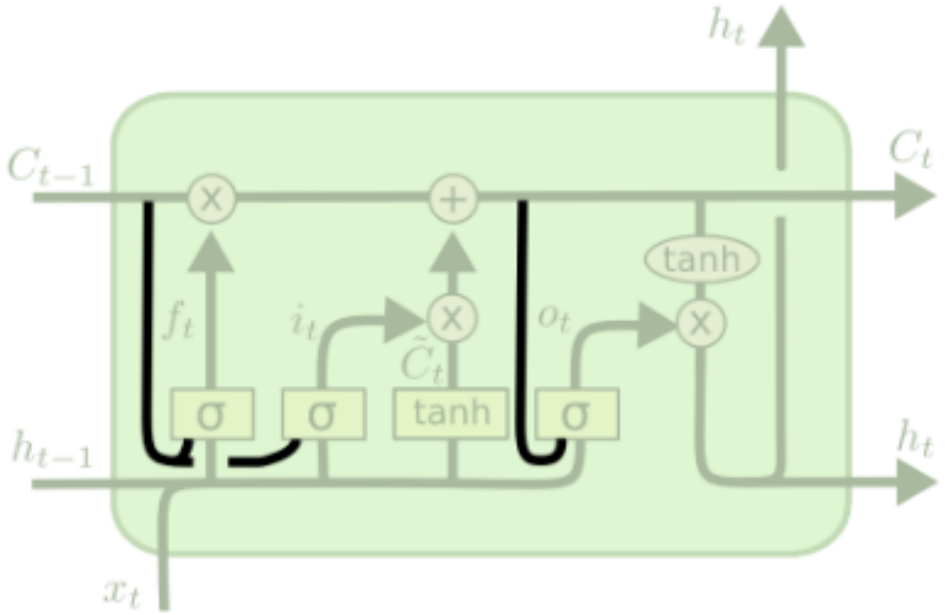
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

- The output will be based on the on our cell state, but will be a filtered version
- First, a sigmoid layer decides which parts of the cell state we are going to output
- The cell state through tanh (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid layer so that we can output the parts we decided to

Variants of Long Short Term Memory (LSTM)

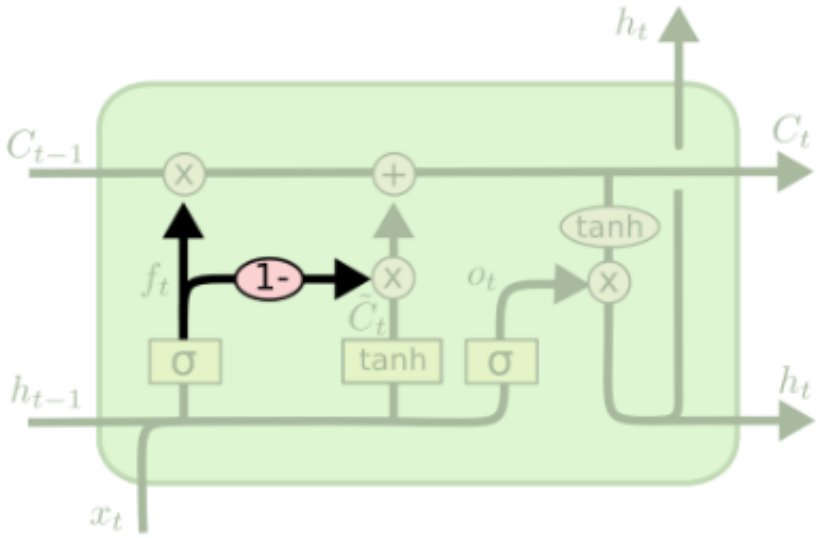
- By Gers & Schmidbuber (2000) by adding “peephole connections”



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

Variants of Long Short Term Memory (LSTM)

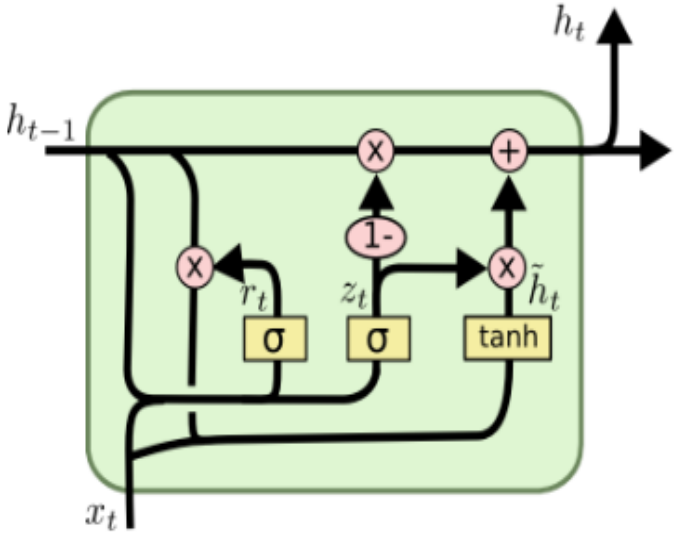
- Instead of separately deciding what to forget and what we should add new information to, we make those decisions together



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

Variants of Long Short Term Memory (LSTM)

- A slightly more dramatic variation on the LSTM is the Gated Recurrent Unit, or GRU, introduced by Cho, et al. (2014)
- It combines the forget and input gates into a single "update gate"
- It also merges the cell state and hidden state, and makes some other changes
- The resulting model is simpler than standard LSTM models, and has been growing increasingly popular



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Applications of Long Short Term Memory (LSTM)

Natural Language Processing

LSTM models are widely used for language modeling, text classification, machine translation, and sentiment analysis

Speech Recognition

LSTM can be trained to recognize and transcribe speech, enabling applications like voice assistants and transcription services

Time Series Prediction

LSTM can be accurately forecast future values in time series data, making it valuable for financial predictions, weather forecasting and so on

Limitations of Long Short Term Memory

- **Computational Complexity and High Resource Usage:** LSTMs have an intricate architecture with multiple gates (input, forget, output) and memory cells, which makes them computationally expensive and slow to train compared to simpler models. They require significant memory and processing power, making them less suitable for large-scale, real-time applications

- **Limited Parallelization:** LSTMs process input data sequentially, one time step at a time, preventing parallelization across time steps

- **Challenges with Extremely Long Sequences:** While LSTMs are designed to address the vanishing gradient problem of traditional RNNs, they can still struggle to maintain information over very long dependencies

- **Overfitting on Small Data:** Due to their high parameter count, LSTMs are prone to overfitting, especially when trained on small datasets

- **Training Instability:** LSTM training can be unstable, prone to exploding gradients (despite the gates) and sensitive to initial hyperparameters, which requires extensive tuning and regularization, such as gradient clipping

- **Lack of Interpretability:** LSTMs are often considered "black box" models, making it difficult to understand the logic behind their predictions

REFERENCES

1. Elman, Jeffrey L. "Finding structure in time." *Cognitive science* 14.2 (1990): 179-211.

2. Bengio, Yoshua, Patrice Simard, and Paolo Frasconi. "Learning long-term dependencies with gradient descent is difficult." *IEEE transactions on neural networks* 5.2 (1994): 157-166

3. Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." *Neural computation* 9.8 (1997): 1735-1780.

4. Gers, Felix A., Jürgen Schmidhuber, and Fred Cummins. "Learning to forget: Continual prediction with LSTM." *Neural computation* 12.10 (2000): 2451-2471.

5. Chung, Junyoung, et al. "Empirical evaluation of gated recurrent neural networks on sequence modeling." *arXiv preprint arXiv:1412.3555* (2014).

6. https://colah.github.io/posts/2015-08-Understanding-LSTMs/


Q&A